

# Wild Animals Intrusion Detection using Deep Learning Techniques

DR.R.S. SABEENIAN<sup>1\*</sup>, N. DEIVANAI<sup>2</sup>, B. MYTHILI<sup>3</sup>

<sup>1</sup>M.E., Ph.D., Professor, Head of Department, Electronics and Communication Engineering, Sona College of Technology, Salem, India.

<sup>2,3</sup>Electronics and Communication Engineering, Sona College of Technology, Salem, India.

\*Corresponding Author

Email ID: [sabeenian@sonatech.ac.in](mailto:sabeenian@sonatech.ac.in), [devi21298@gmail.com](mailto:devi21298@gmail.com), [mythilibala1998@gmail.com](mailto:mythilibala1998@gmail.com)

Received: 12.04.20, Revised: 24.05.20, Accepted: 04.06.20

## ABSTRACT

Crop damage caused by animal attacks is one of the major threats in reducing the crop yield. Due to the expansion of cultivated land into previous wildlife habitat, crop raiding is becoming one of the most antagonizing human-wildlife conflicts. Farmers in India face serious threats from pests, natural calamities & damage by animals resulting in lower yields. Traditional methods followed by farmers are not that effective and it is not feasible to hire guards to keep an eye on crops and prevent wild animals. Since safety of both human and animal is equally vital, it is important to protect the crops from damage caused by animal as well as divert the animal without any harm.

Thus, in order to overcome above problems and to reach our aim, we use machine learning to detect animals, entering into our farm by using deep neural network concept, a division in computer vision. In this project, we will monitor the entire farm at regular intervals through a camera which will be recording the surrounding throughout the day. With the help of a machine learning model, we detect the entry of animals and we play appropriate sounds to drive the animal away. This report specifies various libraries and concepts of convolutional neural networks used to create the model.

**Keywords:** Convolutional Neural Network, Deep learning, Prediction, Training and Validation, Play Sound.

## INTRODUCTION

In agriculture, one of the major social problems that is existing in the present is the damaging of the crops by the wild animals. Wild animal intrusion has always been a persisting problem to the farmers. Some of the animals that act as a threat to the crops are deer, wild boar, moles, elephants, monkeys and others. These animals may feed on crops and also run around the field in the absence of farmer and thus cause damage to those crops. This may in turn results in significant loss in the yield and will cause additional financial protection in order to deal with the aftermath of the damage.

Every farmer, while utilizing his production, should also be aware of the fact that animals are also existing in the same place and that they need to be protected from any probable suffering. This problem must be attended immediately and an effective solution must be created and accomplished. Thus, this project aims to address this problem.

## OBJECTIVE

The main objective of the project is to safe guard the agricultural field from wild animals and also to protect them by driving them away instead of killing. The project also aims to protect human lives from animal attacks. We are using an integrative approach in the field of Deep Learning

to provide a monitoring and repelling system for crop protection against animal attacks.

## LITERATURE REVIEW

Manual ways such as constructing different kinds of fences and using natural repellents are effective but they are not cost efficient. Increasing the manpower is not an ideal solution to this problem [8].

Initially projects were taken up to drive away the animals automatically by using hardware components like controllers and sensors. One such approach is - Camera interfaced to the raspberry pi module. Camera is used to captures an image of wild animal and send captured image to the Raspberry pi module. When image taken by the raspberry pi, it is compared with database image. After comparing images, if the wild animal is detected then it gives commands to GSM (Global System for Mobile communication) module. GSM used to send the message to the owner of the farm. To get the output in the form of audio, connect raspberry pi to the speaker [1]. Disadvantage: The problem here is that, there is more of hardware component which is not cost efficient and its maintenance is also hard.

In recent times, researches are taken to solve this problem using Artificial Intelligence. One of the already proposed idea was WCoHOG (Weighted Co-occurrence of Histogram-Oriented Gradients). WCoHOG is a Histogram oriented gradients-

based feature vector. LIBLINEAR (Library for large linear classification) classifier is used in order to get better accuracy for high dimensional data. The experiments were conducted on two benchmark datasets called Wild-Animal and CamaraTrap dataset. But it is a complex process. This feature vector is supplied to the classifier to detect the animal in a particular window. In sliding window technique different sliding windows are used in order to identify animals of different sizes based upon zoom level of the camera. In the CoHOG (Co-occurrence of Histogram-Oriented Gradients) method, gradient directions are used to calculate feature vector and the magnitude is ignored [2]. One another approach was by identifying the sound of animals and processing it by concepts of signal processing and deep learning which involved more hardware and as well as complex process depending on the database [3].

## OVERVIEW

India Times have mentioned the intrusion of wild animals in private property and as well is in road crossing several times [5]. During these situations, the main aim is to drive away the wild animals automatically without causing loss for human lives as well as animal lives. As manual methods require manpower and system with hardware to detect animals requires maintenance every now and then, the model proposed overcome these difficulties as it involves only maintenance of software code. By using the concept of Deep Learning (DL) with Convolutional Neural Network (CNN) method, the wild animals can be detected when it is captured by the camera. Depending upon the animal detected, appropriate repellent sounds can be played in order to drive away the animal automatically. This project hence, requires a backend code, a 24/7 camcorder and a speaker.

The concepts involved in this model are:

1. Computer Vision – to extract features from images
2. Machine Learning – Supervised learning is needed for the model in order to classify the animals.
3. Deep Learning – This provides the cognitive skill of analyzing the animal detected to a particular class and hence produce the appropriate output, just like how human analyses and this can be achieved with the help of CNN.

The most important packages needed to drive the code are:

OpenCV and 2. Keras: where this both will help in the processing of the data acquired involving all the above concepts in the back-end. Anaconda - Jupyter Notebook platform is used as the coding platform as it is an open source software. Jupyter Notebooks are powerful,

versatile, shareable and provide the ability to perform data visualization in the same environment. It allows data scientists to create and share their documents, from codes to full blown reports. The proposed model uses python language to code as it is easier to understand and requires less time for error correction providing it to be more efficient than other languages. In order to make the model understand the real time data (images in the proposed model) we train and test them several times for better accuracy. Increasing the data for training and validation increases the accuracy of the entire model. The disadvantage of this model is the memory requirement and GPU speed. Increase in dataset will result in increased memory need, resulting in slow speed to train and validate the data. Moreover, frames from the camera's live feed also need memory. Having a dataset of about 2500 images will take around 20 minutes to run 30 epochs (training of data) and produces only 70% accuracy in a HP Notebook Pro system. The proposed system can also be extended to give alert to the farmer or owner via a mobile phone and also share the location of the area where the wildlife intrusion is taking place.

## Devices Used

The devices used in this model are,

1. Personal computer
2. Jupyter notebook
3. Surveillance camera
4. Speaker

PC is the vital component of the model. Using Jupyter notebook, we design the code in the PC and the owner has to just run the code once for the model to get activated.

Jupyter notebook supports many programming languages like C, C++ and python. In this model, python is used. Data cleaning, data visualization and machine learning concepts can be implemented with the help of jupyter notebook. Surveillance camera is used to record the field area live 24/7 and this is fed as an input to the model to detect animals.

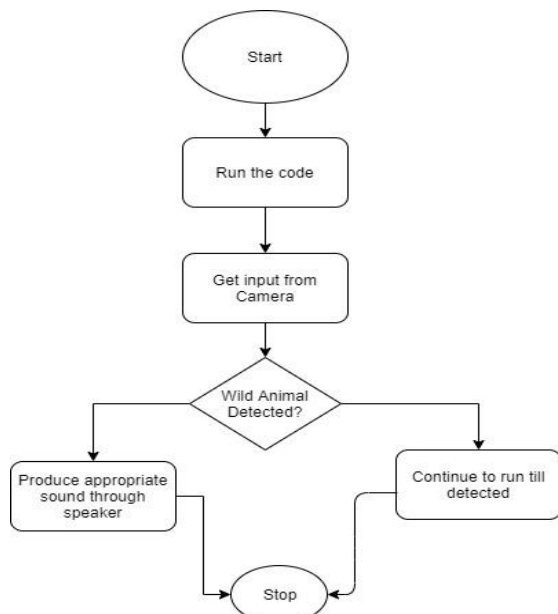
The purpose of speakers is to produce audio output that can be heard loud and clear. This helps to amplify the sound that is received from the model to the field area to drive away animals.

## METHODOLOGY

### Deep Learning Approach

As the above problem stated is still prevailing despite of all the methods taken, we approached the problem using deep learning to drive away the animals automatically. In our project, we used packages like Keras and Playsound to do the pre-processing steps involved and to create an

appropriate output depending on the detected output from the model. Here, the input is received from the CCTV (Closed Circuit Television), the code does the processing and prediction of the frames received from the camera and appropriate repellent sound is played to drive away the detected animal.



**Fig.1: Flow chart of the working model**

#### Dataset

The dataset used to train our model is image dataset. Three classes namely elephant, boar and monkey are taken into consideration as we take these animals as threat to farms and human life in India. The dataset is divided into two categories, one for validation and other for testing.

The training dataset includes:

Monkey – 1000 images  
Elephant – 1023 images  
Boar – 1010 images

The testing dataset includes:

Monkey – 205 images  
Elephant – 203 images  
Boar – 198 images



**Fig.2: Sample of dataset collected for monkey, boar and elephant**

#### Process Involved in Designing

The steps involved in designing the CNN for the proposed model are:

##### Import Libraries

Initially, we import the libraries Sequential, Dense, Convo2D, MaxPooling and Flatten libraries from keras.models and keras.layers packages.

##### Initialize Model

Initialize the model, by calling the constructor – Sequential.

##### Build Convolutional Layer

To add convolution layer to the Neural Network we use the 'add' function along with 'Convo2D'

For the proposed model we use:

```
cnn_model.add(Conv2D(32,3,3,input_shape=(64,64,3)))
```

- 32 denotes the number of feature detectors to be applied.
- 3,3 is the shape of the feature detector.
- input\_shape defines the size of the image and no of channels (color images-3, grey scale images-1). As this is the first layer, we used input\_shape parameter. If we want to perform convolution more than once, this parameter is not necessary from the second cycles.

##### MaxPooling

Next, we add the pooling layer by using add function along with MaxPooling2D.

```
cnn_model.add(MaxPooling2D(pool_size=(2,2)))
```

The pool size represents the filter used in the convoluted image.

##### Efficiency

Again, another set of Convolution and MaxPooling can be performed to increase the efficiency of the model. In the proposed model, we perform the Convolution and MaxPooling process thrice.

##### Flattening

After MaxPooling, we flatten the shape of the image into a single dimension/array. The output from the flatten function is given as the input layer to our future ANN (Artificial Neural Network) model. This step is important because the ANN model accepts input only in single dimension.

##### Training the Dataset Using ANN

The steps involved in training process are:

##### Adding Hidden Layers

To establish full connection, we already have our input layer from the Convolution Network, next we have to add hidden layers for processing the image by the hidden layer neurons. Increasing hidden layers increases the accuracy of the output.

In our project, we add only one hidden layer by using the Dense library

```
cnn_model.add(Dense(init="random_uniform",
activation="relu",output_dim=250))
```

- Init is random uniform here, which is one of the weight optimization strategy.
- The activation function used here is relu to omit the negative values.
- The output dimension is the number of outputs from the hidden layer.

### Final Output Layer

After the hidden layer, final output layer is added.

```
cnn_model.add(Dense(init="randomuniform"
activation="softmax", output_dim=3))
```

Here the activation function used is 'softmax' for categorical data as the output consists of more than two classes (output dimension = 3). 'Sigmoid' activation function is used if our output is binary (consisting of two classes only).

### Compiling the Model

After adding all layers, compile the model.

```
cnn_model.compile(optimizer="adam",loss=
"categorical_crossentropy", metrics=["accuracy"])
```

- Optimizer - adam which is one of the weight optimization strategy.
- Loss- categorical\_crossentropy to calculate the error.
- Metrics – accuracy to calculate the model performance.

### Image Transformation

In order to avoid over fitting of the various images, we need to transform the image. So, use ImageDataGenerator to apply pre-processing steps to the images. For both training and testing dataset, we use pre-processing features like

- Rescaling – 1. /255 is used to apply feature scaling to get values in range of 0 to 1
- Shear\_range – 0.2. This value is to performs geometrical operation to move images in the particular angle.
- Zoom range and horizontal flip. By using all these pre-processing features, we obtain different images with variations and hence avoid overfitting.

### Importing Data and Applying Transformation

Call the 'flow\_from\_directory' function which helps to import data and apply transformations simultaneously. flow\_from\_directory ('file path', target\_size=(150,150), batch\_size=32, class\_mode='categorical')

The arguments that has to be passed are:

- The file path where the training/testing dataset is present.
- Target size – size of the image (we used 64x64 dimension)
- Batch size – as we use mini batch gradient descent to update the weights, we give batch

size i.e. the number of rows after which the weights are updated

- Class mode - categorical as we have more than two classes (boar, elephant and monkey)

Perform this function for both training and testing datasets.

### Class Indices

Use of class\_indices to know the index of each class. This will be helpful when we check the prediction of the testing data. In the proposed model, the indices for the 3 classes is defined as {boar: '0', elephant: '1', monkey: '2'}

### Training and Testing of Dataset

Call the fit\_generator function to train and test the model simultaneously, to provide validation accuracy. The arguments passed are,

```
x_train, steps_per_epoch = number of images in
training dataset/batch_size, epoch=number of
epoch i.e. number of times the model should be
trained, validation_data= x_test,
validation_steps=number of images in testing
dataset/batch_size.
```

The epoch will run and finally we get the validation accuracy and loss as output from this function which denotes how well our model is trained (depending on validation accuracy) and the error produced by the model. More number of epoch will increase the accuracy of the output. In this project we did 40 epochs and received a validation accuracy of 98% and loss of 0.0762.

### Save the Model

Saving the model as 'mymodel.h5' using model.save function.

Thus, the training and testing dataset undergoes the above processes and the images from this model are compared with the live images from the camera to predict whether the animal is entering the field or not. So, for doing this, we again have to load this model to compare it with the frames from camera.

### Designing the Driver Code

The steps involved are:

Loading and Compiling the Saved Model

To get the frames from the live camera, first load the model in order to compare the frames with the trained model.

For that, first we import load\_model, numpy and cv2(OpenCV). Next, load the model that was saved earlier (Eg: 'mymodel.h5'). Then compile the model by giving the optimizer as adam, loss as categorical\_crossentropy and metrics as accuracy similar to the way we compiled the full connection.

### Prediction

To transform the image, resize the image into particular dimension.

In the proposed model, we create a function `detect(frame)` to resize the image so that we can call the function at appropriate places. We use try and except method, wherein in try, we resize the image into 64x64 dimension and expanding the dimension. If the ranges of the image are greater than one, then we rescale it to range 0 to 1 and we predict the output using 'predict' function. Then, to know the class in which the image belongs, we use `predict_classes`.

Shape not found is printed if there is an Attribute Error i.e. if my testing data/frame doesn't match the training set we get this indication.

#### Giving Test Image to Check the Model

To know whether the prediction is right or not, initially give a testing image to find whether it belongs to the appropriate class or not.

In the proposed model the class indices are {boar : '0', elephant: '1', monkey: '2'}

When a boar image is given to test, the output will be [0] and if my test image is a goat, then it should return 'shape not found'.

For this, we have to read the image using `imread` and call the `detect` function.

#### Video Input and Prediction

To detect the animals in the live video, we have to divide it into frames. In order to check the correctness of the model, a sample video is taken and we create a directory to save all the frames that is created. Frames are named automatically and then we read each frame using `imread` and we call the `detect` function to find whether there is an animal in the particular frame. Then, we give

the appropriate sound according to the animal detected by importing `playsound`. This is achieved by using else-if case by giving the class indices as condition. In the proposed model, crackers, high pitch sounds and gun shots were used to drive the particular animal away. After every frame is read from the live video the space and all the windows are released at once in order to save memory.

## RESULTS AND DISCUSSIONS

Thus, the proposed model will train the image dataset of monkey, boar and elephant by establishing Convolutional Neural Network and this model is saved. The saved model will run on the driver code in order to compare the trained images with the new test images from the live capture. If anyone of the animals that is trained is detected during the live capture, a repelling sound is played in order to drive away the animal with the help of speakers.

In order to check the correctness of the model, different test images are given and their classes are detected.

```
In [19]: x_train.class_indices #to know how the datas are assigned
```

```
Out[19]: {'boar': 0, 'elephant': 1, 'monkey': 2}
```

**Fig.3: Class Indices**

```
=====] - 84s 226ms/step - loss: 0.8531 - acc: 0.9815 - val_loss: 0.8414 - val_acc: 0.9877
```

```
=====] - 82s 219ms/step - loss: 0.8658 - acc: 0.9759 - val_loss: 0.8762 - val_acc: 0.9861
```

**Fig.4: Final Validation Accuracy Results**

```
In [9]: frame=cv2.imread(r"C:\dataset\test\boar\ba356f267a288e62418ec6a2d42d390.jpg")
data1=detect(frame)
[[9.6150213e-01 3.8495053e-02 2.8406132e-06]]
[0]

In [13]: frame=cv2.imread(r"C:\dataset\test\elephant\800_7978637371660e5052178e080bf2ab29.jpg")
data2=detect(frame)
[[5.4536948e-05 9.9991906e-01 2.6307334e-05]]
[1]

In [14]: frame=cv2.imread(r"C:\dataset\test\monkey\8b73f565eca8ff4d.jpg")
data3=detect(frame)
[[3.0192099e-05 1.5915020e-07 9.9996908e-01]]
[2]

In [21]: frame=cv2.imread(r"C:\Users\USER\Desktop\human.jpg")
data4=detect(frame)
shape not found
```

**Fig.5: Testing model with test images**

Fig. 5. Shows the correctness of the model by checking all the possible cases with test images.

```
# Read the video from specified path
cam = cv2.VideoCapture(r"C:\Users\USER\Downloads\boar1 video.mp4")
```

**Fig.6: Capturing the real time data**

Command in Fig. 6. Is used to capture the real time data and in this case we have taken boar as example

```

Creating....data/frame0.jpg
[[9.6198773e-01 3.8012177e-02 6.9924882e-08]]
[0]
Creating....data/frame1.jpg
[[9.6200889e-01 3.7990954e-02 6.9878613e-08]]
[0]
Creating....data/frame2.jpg
[[9.6204323e-01 3.7956662e-02 6.9782992e-08]]
[0]
Creating....data/frame3.jpg
[[9.620427e-01 3.795728e-02 6.971519e-08]]
[0]

```

Fig.7: Output Achieved

So, when the boar is detected during capture, we get the audio output (appropriate repelling sound) as well as the class indices of boar (i.e. {0})

### CONCLUSION

The problem of crop vandalization by wild animals has become a major social problem in current time. In other words, while utilizing his/her crop production, every farmer should be aware and take into consideration the fact that animals are living beings and need to be protected from any potential suffering. It requires urgent attention and an effective solution. Thus, this project carries a great social relevance as it will help farmers in protecting their fields and save them from significant financial losses and will save them from the unproductive efforts that they endure for the protection of their fields.

### ACKNOWLEDGMENT

The authors wish to thank Dr.R.S. Sabeenian M.E., Ph.D., Professor, Head of the Department (ECE) who have helped in the completion of the project.

### REFERENCES

1. Santhiya S, Dhamodharan Y, Kavi Priya NE., Santhosh CS and Surekha M. 'A Smart Farmland Using Raspberry Pi Crop Prevention and Animal Intrusion Detection System '. International Research Journal of Engineering and Technology (IRJET), 2018; 05(03)
2. Andavarapu N and Vatsavayi VK. Wild-animal recognition in agriculture farms using W-COHOG for agro-security. International Journal of Computational Intelligence Research, 2017; 13(9): 2247-2257.
3. Duhart C, Dublon G, Mayton B, and Paradiso J. 'Deep Learning Locally Trained Wildlife Sensing in Real Acoustic Wetland Environment'. In Thampi SM, Marques O, Krishnan S, Cuonzo D and Kolekar MH (eds.), Advances in Signal Processing and Intelligent Recognition Systems, 2019: 3-14.
4. Jordan MI and Jacobs RA. Hierarchies of adaptive experts. 'Advances in Neural Information

Processing Systems', 1992: 985-992

5. <https://www.indiatimes.com/news/india/in-the-battle-between-man-vs-wild-highways-railway-lines-emerge-as-new-challenges-375504.html>
6. <https://adventuresinmachinelearning.com/neural-networks-tutorial/>
7. <https://github.com/tensorflow/models>
8. [https://digitalcommons.unl.edu/cgi/viewcontent.cgi?article=2189&context=icwdm\\_usdanwrc](https://digitalcommons.unl.edu/cgi/viewcontent.cgi?article=2189&context=icwdm_usdanwrc)